700105 Simulation & Concurrency

UNIVERSITY of HULL

The story so far

The story so far

- Updating State
  - Calculating **(linear)** forces
    - Gravity
  - Calculating new velocities and positions **(particle/projectile motion)**
    - Integration methods (Euler, Semi-Implicit Euler, Verlet, Runge-Kutta)
- Collisions - Geometric Objects (Shapes)
  - Primitives (point, line, line segment, plane, sphere, cylinder, capsule, cuboid, triangle, convex hull)
  - Methods (Inside-Outside, Intersects, Contains, SAT, GJK)
- Collision Response
  - Calculating new **(linear)** velocities or forces after collision
  - Conservation of **(linear)** momentum

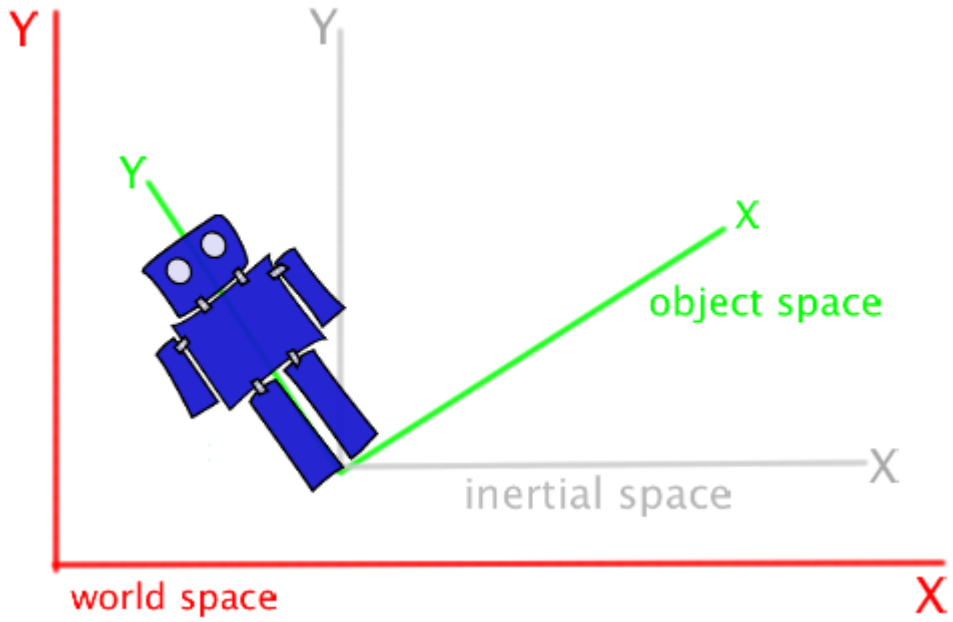
Rotating things

- This week we will look at orientation, angular displacement and angular velocity
- Then by considering nested spaces we can treat linear motion of the centre of mass and orientation around the centre of mass separately

Orientation

Space Definitions

- World space
- Inertial Space
  - The position of an object in world space
- Object Space
  - The position and rotation of an object in world space
- This is an example of nested coordinate space



Orientation

- Orientation is how an object is ***oriented*** in space
- Direction is the direction and object is pointed in
  - This is subtly different to orientation, because you can change the orientation of an object without changing the direction by rotating it around the direction
- Rotation
  - Just as Position is given relative to some other known position (like the origin) rotation gives the orientation of an object relative to some other known orientation
- Angular Displacement
  - The amount of rotation to get from one orientation to another

Matrix Form

- We can represent a rotation as a 3x3 matrix
- Express the 3 basis vectors (unit x, y and z) in the rotated space
- Basis vectors x (1, 0, 0), y(0, 1, 0) and z(0, 0, 1) become rotated vectors

$$R = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}$$

Basis vector	Rotated vector
1, 0, 0	a, b, c
0, 1, 0	d, e, f
0, 0, 1	g, h, i

Rotation around basis vectors

- Individual rotation matrices can be derived using trigonometry

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}$$

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$R_z(90^\circ)\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} \cos 90^\circ & -\sin 90^\circ & 0 \\ \sin 90^\circ & \cos 90^\circ & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

- Clockwise rotation around z axis
- Notice that as z doesn’t change the last row and column look like the identity matrix

Pros and Cons

- Same format used to rotate vectors
- Same format used by graphics APIs
- Easy to concatenate multiple angular displacements
- Can calculate inverse of a rotation

- More memory could be significant if you have a model with lots of joints
- Difficult to understand by observation
- Six degrees of redundancy so can become malformed by a skew, scale or shear (for example)

Homogenous Space (Reminder)

- So far you've probably used a 4x4 matrix to represent the transformation of vectors from one space to another
- This is homogenous because it allows us to apply rotation and translation in one operation
- Translation is actually a 4D shear projected onto 3D space with w = 1

$$\mathbf{M} = \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Some combination of rotations is represented in the top left 3x3 elements
- Scaling is also incorporated into the diagonal (a_{11}, a_{22} , a_{33})

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix}$$

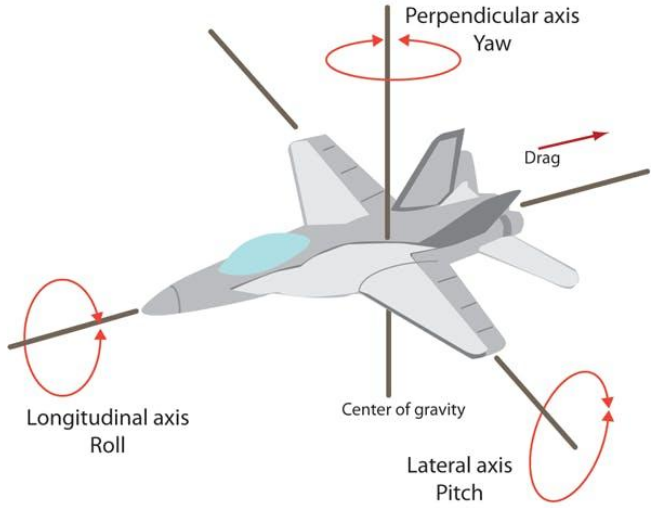
$$R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}$$

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{M} = \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Euler Angles

- Euler angles express a rotation in terms of three ordered rotations around three perpendicular axes
  - Typically the three cardinal axes
  - Typically roll -> pitch -> yaw in aerospace



$$R = R_z(\gamma)\, R_y(\beta)\, R_x(\alpha) = \overset{\text{roll}}{\begin{bmatrix} \cos\gamma & -\sin\gamma & 0 \\ \sin\gamma & \cos\gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}} \overset{\text{pitch}}{\begin{bmatrix} \cos\beta & 0 & \sin\beta \\ 0 & 1 & 0 \\ -\sin\beta & 0 & \cos\beta \end{bmatrix}} \overset{\text{yaw}}{\begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha \\ 0 & \sin\alpha & \cos\alpha \end{bmatrix}}$$

$$= \begin{bmatrix} \cos\beta\cos\gamma & \sin\alpha\sin\beta\cos\gamma - \cos\alpha\sin\gamma & \cos\alpha\sin\beta\cos\gamma + \sin\alpha\sin\gamma \\ \cos\beta\sin\gamma & \sin\alpha\sin\beta\sin\gamma + \cos\alpha\cos\gamma & \cos\alpha\sin\beta\sin\gamma - \sin\alpha\cos\gamma \\ -\sin\beta & \sin\alpha\cos\beta & \cos\alpha\cos\beta \end{bmatrix}$$

- This is another example of nested coordinate space
- Note roll -> pitch -> yaw is common in aerospace, but yaw -> pitch -> roll is more common in games

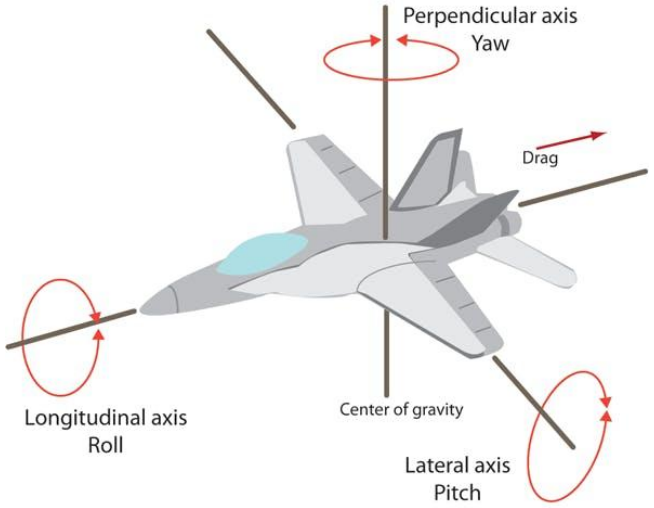
Pros and Cons

- Easy to use
- Small memory footprint
- No invalid set of Euler angles

- An orientation from Euler angles is not unique
  - This means that different Euler angles can result in the same rotation (e.g. 360, 720)
  - Cannot decompose a given orientation into its Euler angles because there are many possibilities
- Difficult to interpolate
- Gimbal lock
  - This results from non-uniqueness
  - Loss of a degree of freedom by rotation one axis onto another

Gimbal Lock

- Assume roll -> pitch -> yaw
  - If you pitch 90 degrees – because pitch happens inbetween yaw and roll one axis is rotated onto the other, which is stationary
- The important concept is that RxRy != RyRx



Quarternions

- Representing rotations as a point on a unit 4d hypersphere projected into 3d space
- Has 1 real component (half the rotation angle) and 3 imaginary components

$$q = q_0 + iq_1 + jq_2 + kq_3$$

$$\begin{aligned}
i^2 = j^2 = k^2 &= -1 \\
i = jk &= -kj \\
j = ki &= -ik \\
k = ij &= -ji.
\end{aligned}$$

Pros and Cons

- **Interpolation is easy**
- Fast concatenation and inversion
- Fast conversion to and from matrix form
- Four floating points

- Difficult to understand
- More prone to accumulating floating point error than matrices

Quarternions

- More on quarternions later
- For now here are some excellent resources
- GDC Talk 1
- GDC Talk 2
- 3Blue1Brown Quarternions
- 3Blue1Brown Visualizing Quarternions
- https://eater.net/quaternion

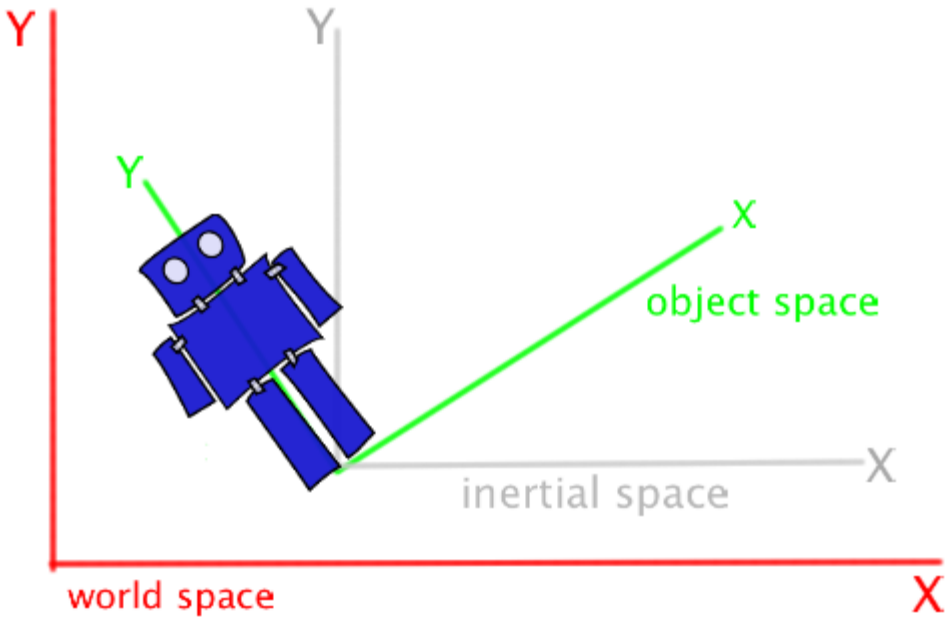
Angular Velocity and Momentum

Projectile/Particle Motion

- So far we have considered points to be particles that only move in a straight line
- When an arbitrary object moves through space with just linear velocity each point in that object moves in parallel with every other point
- If an object rotates it rotates around an axis that passes through the centre of mass of an object
  - This is true unless there are additional constraints (such as a hinge)
- This also means that the centre of mass moves linearly

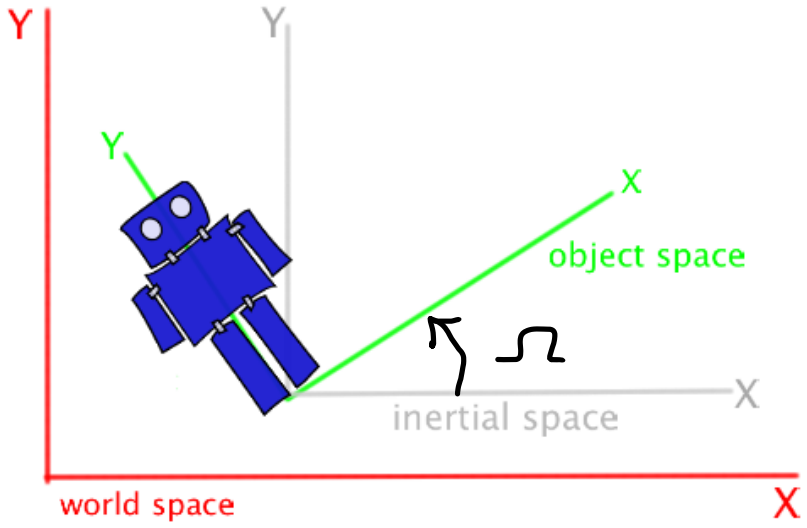
Nested coordinate spaces

- For physics, consider world space -> inertial space to be the linear motion of the centre of mass of an object
- Inertial space -> object space is the orientation of the object in inertial space



Some definitions

- Ω is angular displacement (radians)
- ω is rate of change of angular displacement (angular velocity, rad/s)
  - $\omega = \mathrm{d}\,\Omega/\mathrm{d}\,t$
- α is rate of change of angular velocity (angular acceleration. rad/s^2)
  - $\alpha = \frac{\mathrm{d}\omega}{\mathrm{d}t} = \frac{\mathrm{d}^2\Omega}{\mathrm{d}t^2}$
- $\omega = \int \alpha \, \mathrm{d}t$
- $\Omega = \int \omega \, \mathrm{d}t$
- $\omega \, d\omega = \alpha \, d\Omega$
- These are very similar to our linear equations



More equations

- We can derive similar angular equations to linear equations for constant angular acceleration
- $\omega_2 = \omega_1 + \alpha t$
- $w_2^2 = \omega_1^2 + 2\alpha(\Omega_2 - \Omega_1)$
- $\Omega_2 = \Omega_1 + \omega_1 t + \frac{1}{2} \propto t^2$

Takeaways

- If we treat the centre of mass linearly we can separate out linear and rotational motion
- Next time we will look at applying angular acceleration and torque

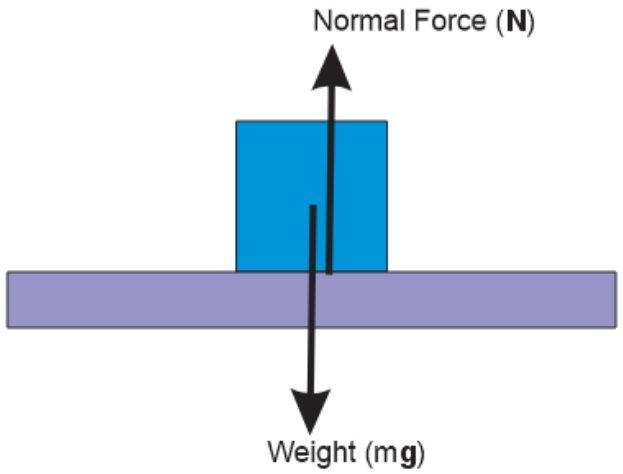
Time to “Code”

Canvas Workshop 5.1 – Start creating tests for angular velocity

Friction Forces

Friction

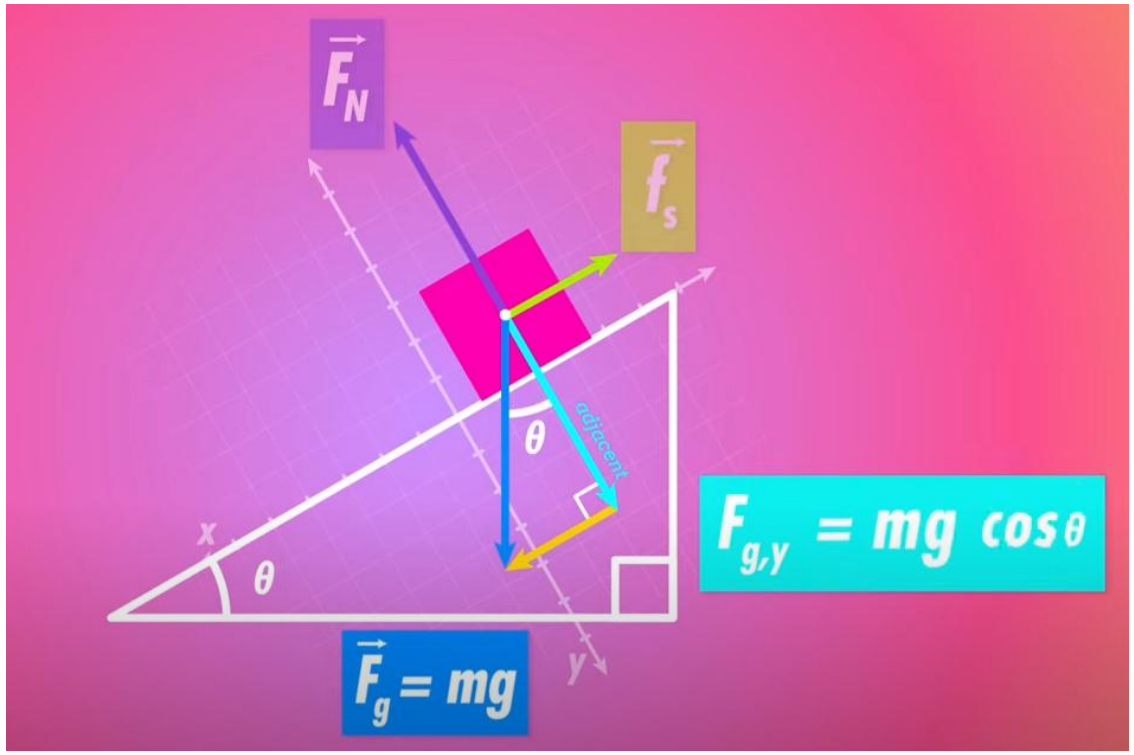
- Friction is a term for lots of forces that resist motion – usually because of some interaction between contacting surfaces because one object is sliding over the other
- Friction is always tangential to the contacting surfaces at the point of contact
- Friction is a function of the normal force N (i.e weight) and the surface roughness (similar to elasticity)



Static and Dynamic Friction

- Static friction is the amount of force to start a stationary object moving
- $\mathbf{F}_{fmax}=\mu_s\mathbf{N}$
- Dynamic (or kinetic) friction is the amount of resistance on an object that is already moving
- $\mathbf{F}_{fk}=\mu_k\mathbf{N}$
- μ_s and μ_k depend on the two surfaces
- μ_s is less than or equal to μ_k

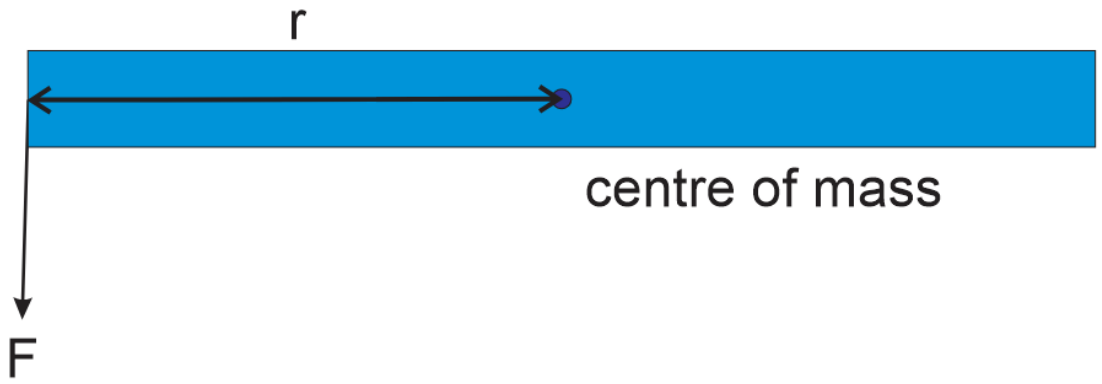
- Note in the diagram the x and y axis have been oriented along the slope



Torque

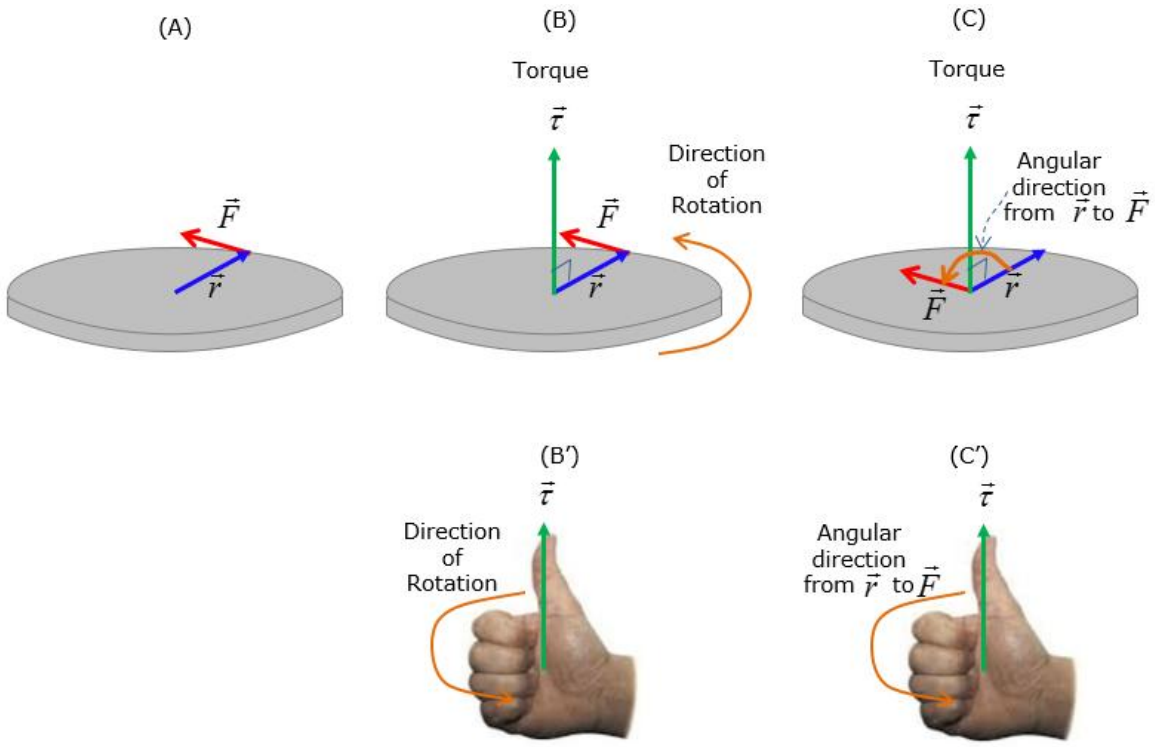
Torque

- Torque is the rotational equivalent to force
  - Sometimes called moments
- Most forces don't act through the centre of mass
  - For example Friction almost always has a component that is perpendicular to the centre of mass
- In the same way that linear forces act through the centre of mass of an object, torque acts at perpendicular to the centre of mass
- Torque is proportional to the perpendicular distance to the centre of mass
- $\boldsymbol{\tau} = r * \mathbf{F}_{\mathbf{perp}}$ (or sometimes $\mathbf{M} = r * \mathbf{F}_{\mathbf{perp}}$)



Scaled axis representation

- **τ** = a * **d**
- Where a is the magnitude of the torque and **d** is the unit axis of rotation according to the right hand rule
- A similar convention is also used for angular velocities
- As a mental model it can also be helpful to think of rotational motion as the accumulation of lots of small linear motions of points in a shape



Door example

- For a door on a hinge the torque acts at the point of rotation (not the centre of mass)
- Consider the force required to open a door if the handle were close to the hinge, versus far away
- We want to apply a smaller force over a larger distance instead of a large force over a small distance

This is dumb

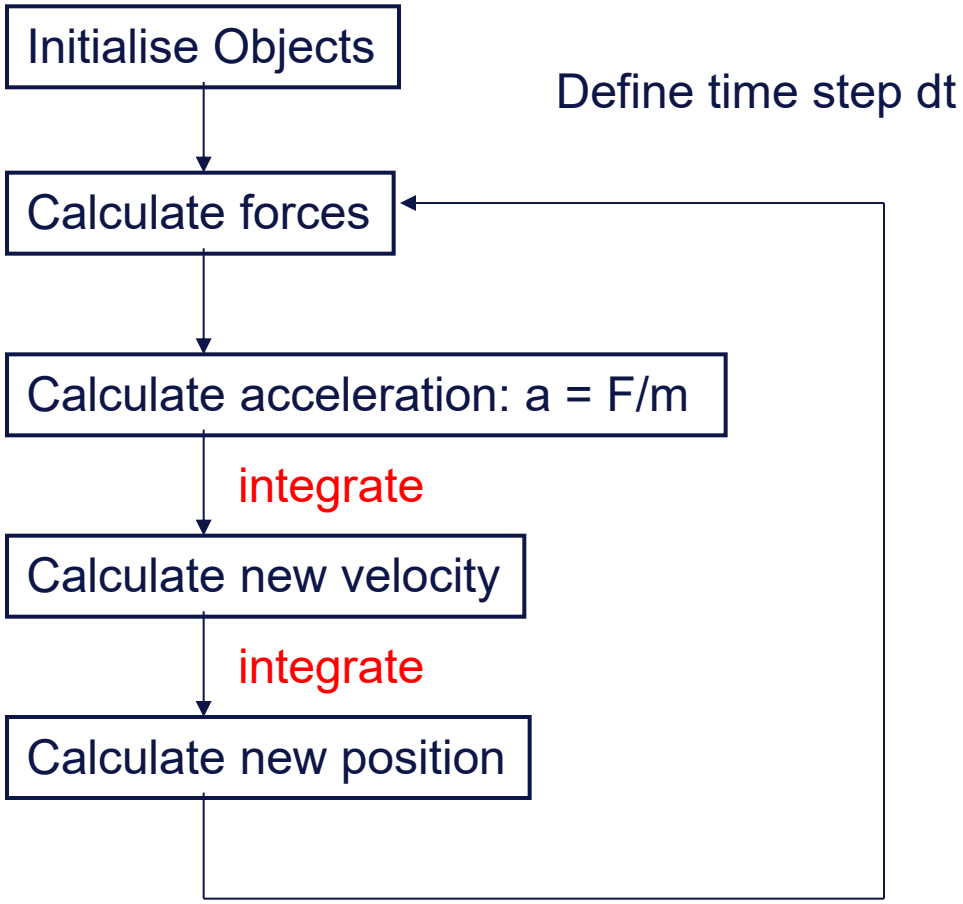
This too

General forces

- Generally forces won’t fall nicely into one of these cases
  - Then we have to use trigonometry to decompose the force into parallel (linear motion) and perpendicular (torque/rotational motion) components relative to the centre of mass

Simulation Loop

- Calculate forces becomes
  - Calculate linear forces
  - Calculate torques



Angular Momentum

Angular Momentum

- Angular Momentum **L** is similar to linear momentum
- $\tau = \frac{dL}{dt}$
- $L = I\omega$
  - Where I is the moment of inertia, which is dependent on a lot of things
  - Remember though Inertia is similar to mass

Moment of inertia

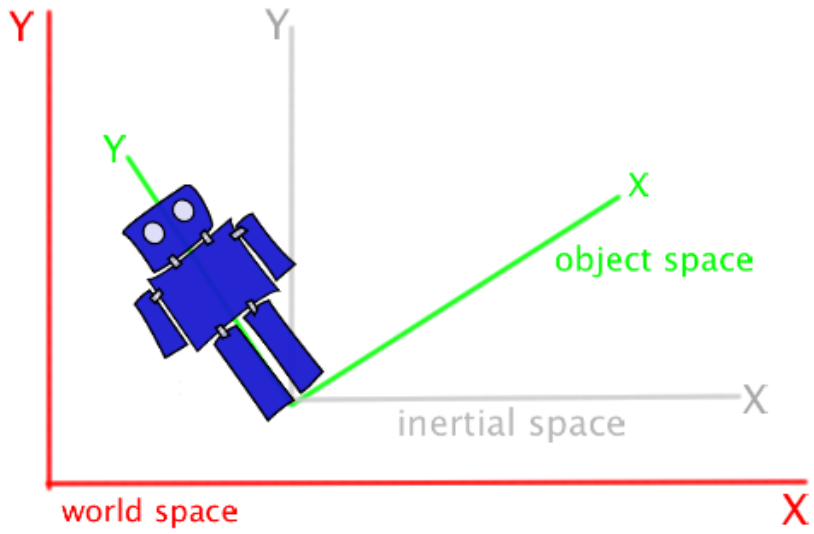
- Inertia is similar to momentum
  - Momentum is the tendency of an object to keep moving (because it is heavy)
  - Inertia is the resistance to change in motion (either to start or stop moving, or change direction)
- In the particle model we only need to consider linear momentum
- The moment of inertia is a representation of the distribution of mass throughout an object
- For rotation inertia can be thought of as the resistance to motion perpendicular to the line to the centre of mass of all the points in the object
- This depends on the shape of an object, as well as the distribution of mass of an object relative to the axis of rotation

Moment of Inertia of spheres

- Solid sphere
  - Rotationally symmetrical, so easy to thing about
  - Uniform distribution of mass
  - $I = 2/5\ mr^2$
- Hollow sphere
  - Rotationally symmetrical, so easy to thing about
  - More mass on the outside than in the middle
  - $I = 2/3\ mr^2$ where r is the outer radius

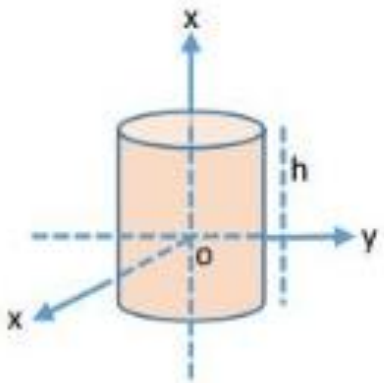
Inertia of asymmetric objects

- Moment of inertia of asymmetric objects requires a more complex representation in the same way that a rotation requires a more complex representation
- We can use an inertia tensor – a 3x3 matrix that represents the moment of inertia along 3 axis of an object



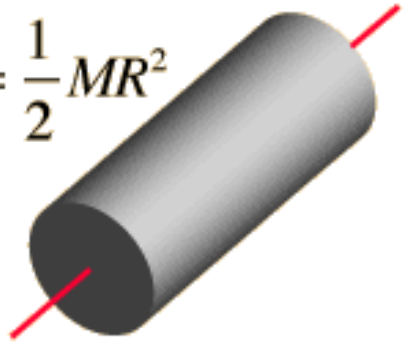
Inertia of a cylinder

$$\{I\} = \begin{bmatrix} \frac{Mh^2}{12} + \frac{MR^2}{4} & 0 & 0 \\ 0 & \frac{Mh^2}{12} + \frac{MR^2}{4} & 0 \\ 0 & 0 & \frac{MR^2}{4} \end{bmatrix}$$



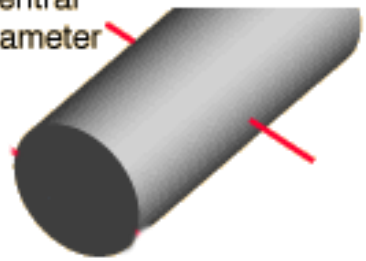
Central axis

$$I = \frac{1}{2} MR^2$$



$$I = \frac{1}{4} MR^2 + \frac{1}{12} ML^2$$

Central diameter



Inverse Inertia Tensor

- Recall that F = ma
- $\tau = \frac{dL}{dt}$
- $L = I\omega$ (L is angular momentum – remember I is a representation of distribution of mass)
- We want to calculate ω so it is useful to store I^{-1} (the inverse inertia tensor)

Optimisation - Inertia Tensor in World Coordinates

- Typically Torque and angular acceleration are calculated in world coordinates – not object coordinates
- It is more useful to store the inverse inertia tensor in world coordinates
- Need to calculate this value each frame
- I recommend moving from the easier to understand, sub-optimal solution to the more optimal solution to ensure correctness

Time to “Code”

Canvas Workshop 5.1 Applying torque (if there is time)